

AN1.10 - Permissions / Guardrails Deepening (A8)

Sprint: AT24 AI Agents - Agent Framework Foundation **Step:** A8 - the central authorization decision boundary **Depends on:** A1 permission contract, A2 tool metadata, A4 runtime + LimitEnforcer, A6 integrity, A7 memory (all closed) **Gate:** G08 - one authoritative authorization decision; permission AND autonomy are separate controls.

A8 consolidates - it does not create a parallel governance system. One `AuthorizationService` consumes the existing policies (A1 grants, A2 tool metadata, A4 `checkLimits`, the A1 `LIVE_EXECUTION` double gate) and returns one deterministic `AuthorizationDecision` recording every gate.

1. What was built

1.1 Contract - additive to `permission-contract.ts` (AF-v1 unchanged)

- `PERMISSION_AUTONOMY_FLOOR` - the minimum agent autonomy required to *hold* a permission: `CAN_GENERATE_SIGNAL` -> 1, `CAN_CREATE_ORDER` -> 3, `CAN_EXECUTE_ORDER` -> 3. Holding one above your autonomy is an invalid configuration.
- `PROHIBITED_PERMISSION_COMBINATIONS` - permission sets a single agent must never hold together (`CAN_GENERATE_SIGNAL` + `CAN_EXECUTE_ORDER`, `CAN_CREATE_ORDER` + `CAN_GENERATE_SIGNAL` - each composes into an unsupervised trading loop). `prohibitedCombosViolated(granted)` returns the violations.

1.2 `services/agent-framework/authorization/authorization-service.ts`

`AuthorizationService.authorize(input)` -> `AuthorizationDecision`.

```
input = { definition, request, registry, limitContext, creditEstimate, liveExecution? }
output = { outcome: "allow" | "deny" | "needs_approval",
  intent?: AuthorizedToolIntent,
  denialCode?: AuthorizationDenialCode,
  breachedLimit?, message?,
  checks: AuthorizationCheck[] // EVERY gate, in order, pass/fail }
```

Gates, in order - every one recorded, ALL must pass:

#	gate	denial code
1	malformed_request - toolId present; request carries no permissions/granted/autonomy/autonomyLevel/authorizedBy field	malformed_request
2	tool exists in the registry (default-deny)	unknown_tool
3	tool status === "active"	tool_disabled
4	tool executionMode in {sync, resumable}	execution_mode_unsupported
5	agent autonomy ceiling - <code>autonomyLevel <= min(MAX_CONFIGURABLE_AUTONOMY_V1, registry type cap)</code> (re-checked from the <i>persisted</i> definition - defence in depth)	agent_autonomy_ceiling
6	every held permission's autonomy floor <= agent autonomy	permission_autonomy_floor

#	gate	denial code
7	no prohibited permission combination	prohibited_combination
8	LIVE_EXECUTION = DENY - a tool requiring <code>CAN_CREATE_ORDER/CAN_EXECUTE_ORDER</code> is denied unless <code>liveExecutionEnabled && userAllowlisted && autonomyLevel >= 3</code> (none true in v1)	live_execution_denied
9	permission - <code>evaluatePermission(policy, tool.requiredPermissions)</code>	missing_permission
10	tool autonomy floor - <code>canRunAtAutonomy(agent, tool.autonomyFloor)</code> (separate control from #9)	tool_autonomy_floor
11	resource limits - <code>checkLimits(limitContext, creditEstimate)</code> (consolidated - A4's function, not reimplemented)	resource_limit (+ breachedLimit)

All pass -> allow with an `AuthorizedToolIntent`, unless `tool.autonomyFloor >= 3` -> `needs_approval` (structurally unreachable in v1 - gate 10 denies first - but the decision path exists).

1.3 Consolidation - what A8 replaced / consumed

Concern	Owner	A8
Permission grants + autonomy level	A1 <code>AgentDefinition</code>	reads
Tool capability metadata	A2 <code>ToolDefinition</code> (registry)	reads
Resource limits	A4 <code>checkLimits</code> (LimitEnforcer)	calls (gate 11)
LIVE_EXECUTION double gate	A1 <code>validatePermissionPolicy</code> (write-time) + A8 (authorize-time)	is the second gate
Output integrity	A6	untouched
Memory policy	A7 <code>MemoryGateway</code>	untouched
Scattered A4 <code>authorizer.ts</code>	A4	deleted - the runtime now calls <code>AuthorizationService.authorize()</code> directly; <code>runtime/index.ts</code> re-exports the service

The runtime's `doRunningSlice` previously did a credit pre-check **and** a separate `authorizeToolRequest`. Both are now the single `authorize()` call. A capability/permission denial records an `AgentToolCall` row (audit trail); a resource-limit denial does not (a budget stop - "0 tool calls" stays true).

1.4 The planner / LLM can never grant itself authority

Structural, two ways:

1. `PlannerToolRequest` carries only `{ toolId, input, rationale }` - **no** permission or autonomy field (asserted against the contract source).
 2. `authorize()` reads grants **only** from `definition.permissionPolicy` and `definition.autonomyLevel`. A request object carrying a `permissions / granted / autonomy / authorizedBy` key -> gate 1 denies `malformed_request`.
-

2. G08 proof - `npm run validate:agent-authorization` -> 17 passed, 0 failed

ALLOW: permitted tool + sufficient autonomy + within limits -> allow + AuthorizedToolIntent; every gate recorded, all passed	■
DENY unknown tool -> <code>unknown_tool</code> (default deny)	■
DENY missing permission -> <code>missing_permission</code> ; permission gate passed: <code>false</code>	■
SEPARATE controls - permission allow but autonomy floor not met -> <code>tool_autonomy_floor</code> (permission gate passed: <code>true</code>)	■
SEPARATE controls - autonomy sufficient but permission <code>deny</code> -> <code>missing_permission</code> (autonomy gate never reached)	■
DENY agent autonomy ceiling exceeded (stale/tampered def) -> <code>agent_autonomy_ceiling</code>	■
DENY agent holds <code>CAN_GENERATE_SIGNAL</code> at autonomy 0 -> <code>permission_autonomy_floor</code>	■
DENY <code>LIVE_EXECUTION</code> - a tool requiring <code>CAN_EXECUTE_ORDER</code> , env gates off -> <code>live_execution_denied</code> (<code>live_execution</code> gate passed: <code>false</code>)	■
DENY prohibited combination (<code>CAN_GENERATE_SIGNAL</code> + <code>CAN_EXECUTE_ORDER</code>)	■
DENY resource limit - credit estimate over ceiling -> <code>resource_limit / maxCreditCost</code>	■
DENY resource limit - step count at ceiling -> <code>resource_limit / maxSteps</code>	■
DENY request carrying any authority field (permissions/granted/autonomy/autonomyLevel/authorizedBy) -> <code>malformed_request</code>	■
<code>PlannerToolRequest</code> body contains no authority field (structural)	■
unauthorized memory operation still denied by the <code>MemoryGateway</code> (governance consistent across subsystems)	■
RUNTIME: a tampered persisted definition (<code>autonomyLevel</code> raised to 4 in the DB row) -> run terminates <code>permission_denied / agent_autonomy_ceiling</code> ; the denial step records every gate evaluated	■
RUNTIME: a compliant agent still reaches <code>succeeded</code>	■

2.1 Regression - no gate lost

`validate:agent-contracts` 38/0 - `validate:agent-tools` 20/0 - `validate:agent-run-persistence` 17/0 - `validate:agent-runtime` 9/0 (one denial-code assertion updated: `autonomy_floor` -> `tool_autonomy_floor`) - `validate:agent-supervisor` 11/0 - `validate:agent-integrity` 21/0 - `validate:agent-memory` 19/0 - `validate:agent-authorization` **17/0**.

3. TypeScript

`npx tsc --noEmit: 0 errors` in `services/agent-framework/**`, `types/agent-framework/**`, `scripts/validate-agent-*.ts`. Repo-wide 77, all in the stale generated `.next/dev/types/validator.ts` (gitignored, pre-existing).

4. Files changed

Added (3):

```
frontend/services/agent-framework/authorization/authorization-service.ts
frontend/services/agent-framework/authorization/index.ts
frontend/scripts/validate-agent-authorization.ts
```

Modified (4):

```
frontend/types/agent-framework/permission-contract.ts + PERMISSION_AUTONOMY_FLOOR, PROHIBITED_PERMISSION_C
frontend/services/agent-framework/runtime/agent-runtime.ts authorization is now the single AuthorizationSe
frontend/services/agent-framework/runtime/index.ts re-exports authorizationService (was authorizeT
frontend/scripts/validate-agent-runtime.ts denial-code assertion updated
frontend/package.json + "validate:agent-authorization"
```

Deleted (1):

```
frontend/services/agent-framework/runtime/authorizer.ts superseded by the central AuthorizationService
```

Not touched: Prisma schema (no migration), contracts A1-A7 behaviour, the tool gateway's defence-in-depth re-checks, A6 integrity, A7 memory gateway, legacy `services/agents/*` + `/dashboard/agents`. No API routes.

5. Non-goals honoured

- No live-execution tool introduced. `LIVE_EXECUTION = DENY` double-denied (write-time parser + authorize-time gate).
- No new parallel governance system - one decision boundary consuming the existing policies.
- No approval UX/workflow - `needs_approval` is a decision *outcome*; the runtime maps it to `permission_denied / approval_required` (no machinery), and it is structurally unreachable in v1 anyway.
- No new agents / tools / engines / UI.

6. The hierarchy is now real

```
Agent Definition (A1)
-> Permission Policy + Autonomy Level (A1)
-> Run Limits snapshot (A4)
-> Supervisor plan (A5, intents only)
-----
-> AuthorizationService.authorize() (A8) <- permission AND autonomy AND
    |                                     limits AND live-exec AND combos,
    |                                     one deterministic decision
    |
    v
AuthorizedToolIntent
-> ToolGateway (A2, defence-in-depth re-check)
-> Executor -> existing AT24 service
-> Evidence -> Output -> Integrity gate (A6)
-> Durable Run (A3)
```

7. Status

A8 complete. Authorization is one deterministic, fully-audited decision boundary. Permission and autonomy are enforced as separate controls that must both pass. `LIVE_EXECUTION` is denied at two independent layers. The planner/LLM cannot inject a grant. A tampered persisted definition is caught at authorization time.

Next per AN1.1 sec 7: A9 (Credit ledger - real accounting to replace the placeholder estimates), A10 (Evaluation + Observability), then A11-A13 (the three real agents).

G08 review requested.

End AN1.10.